

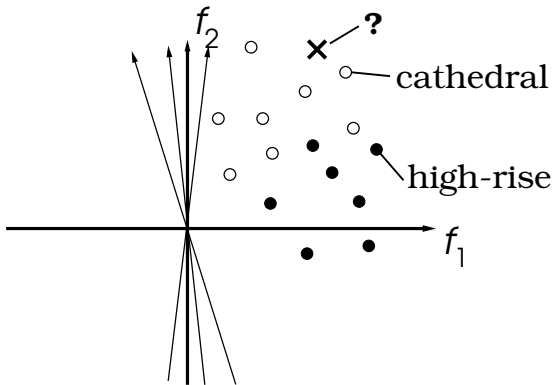
Deep Learning in Image Processing and Vision PART 2

Yoshua Bengio & Roland Memisevic

University of Montreal

September 27, 2015

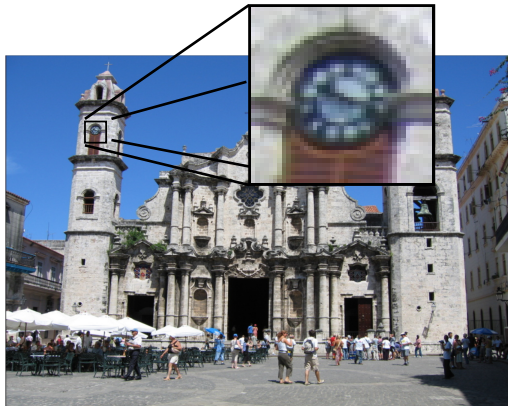




“It’s the features, stupid!”



“It’s the features, stupid!”



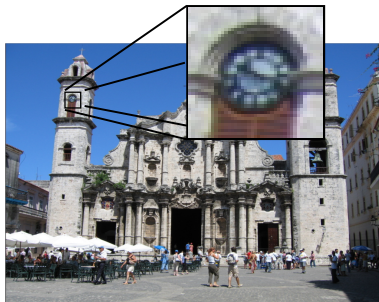
“It’s the features, stupid!”



A common computer vision pipeline before 2012

1. Find interest points.

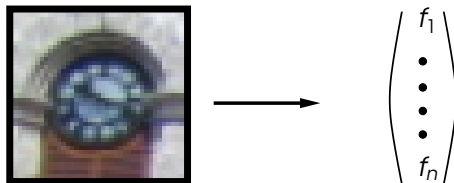
“It’s the features, stupid!”



A common computer vision pipeline before 2012

1. Find interest points.
2. Crop patches around them.

“It’s the features, stupid!”



A common computer vision pipeline before 2012

1. Find interest points.
2. Crop patches around them.
3. Represent each patch with a sparse local descriptor.

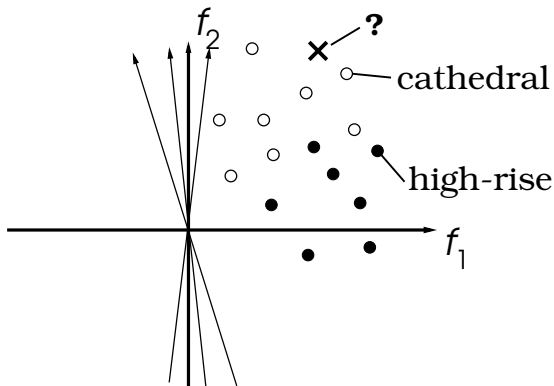
“It’s the features, stupid!”

$$\begin{pmatrix} f_1^1 \\ \vdots \\ f_n^1 \end{pmatrix} + \dots + \begin{pmatrix} f_1^M \\ \vdots \\ f_n^M \end{pmatrix}$$

A common computer vision pipeline before 2012

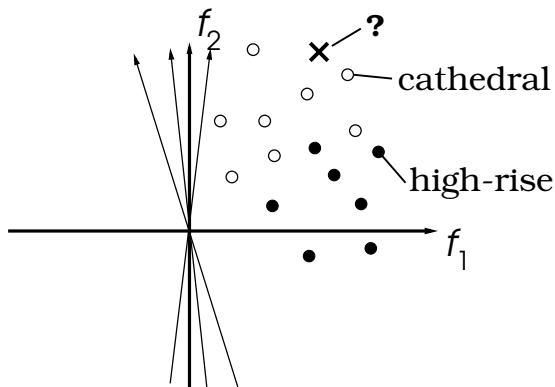
1. Find interest points.
2. Crop patches around them.
3. Represent each patch with a sparse local descriptor.
4. Combine the descriptors into a representation of the image.

“It’s the features, stupid!”



- This creates a representation that even a linear classifier can deal with.

“It’s the features, stupid!”



- This creates a representation that even a linear classifier can deal with.

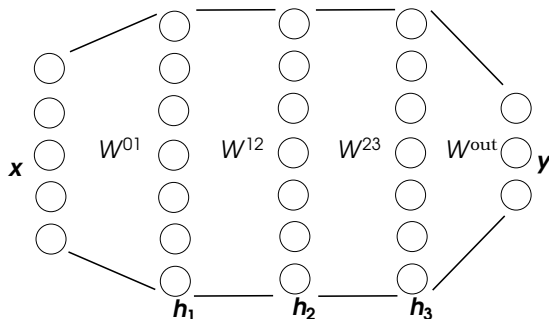
bottom line: **non-linear pipelines are useful**
(aka “*the representation matters*”)

What do good low-level features look like?

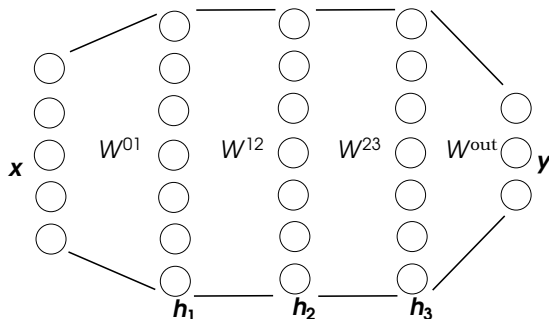


- ▶ Local features that are often found to work well are based on oriented structure (such as Gabor features)
- ▶ These were discovered again and again (also in other areas) and are closely related to the Short Time Fourier Transform.

Neural networks are *trainable* pipelines

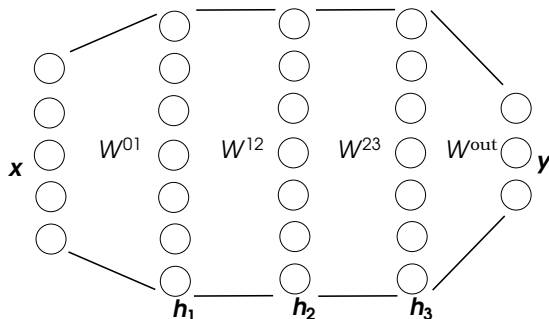


Neural networks are *trainable* pipelines



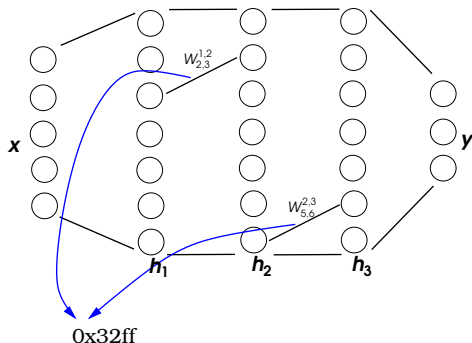
- Learning allows us to harness **training data**
 $(\mathbf{x}_n, \mathbf{t}_n)_{n=1 \dots N}$

Neural networks are *trainable* pipelines



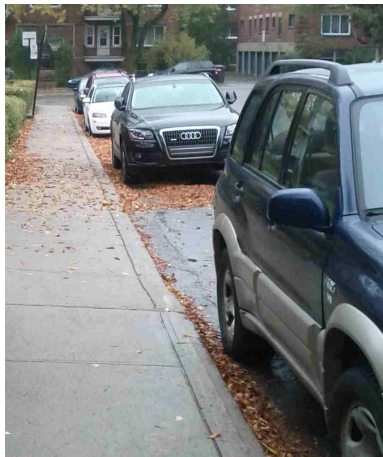
- ▶ Learning allows us to harness **training data**
 $(\mathbf{x}_n, \mathbf{t}_n)_{n=1 \dots N}$
- ▶ Learning allows us to harness **parallelization**

Weight sharing



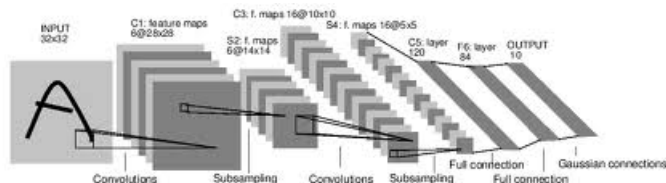
- ▶ Parameters can be shared by having them point to the same memory location.
- ▶ This is a very common way to reduce parameters and encode prior knowledge.
- ▶ The central ingredient in **conv-nets (CNNs)** and **recurrent nets (RNNs)**.
- ▶ *Caveat: It requires long-range communication.*

Translation invariance and locality



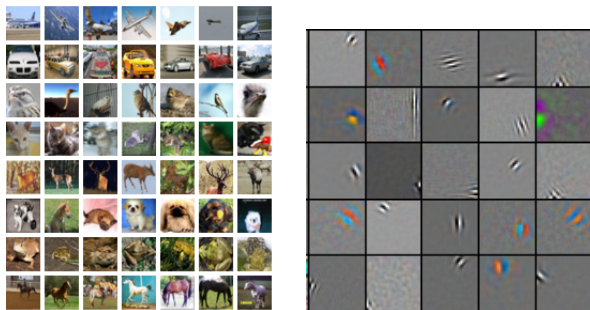
- ▶ Most structure in natural images is **local**
→ Most low-level operations may be based on patches.
- ▶ Most structure in natural images is **position-invariant**
→ We may perform the same set of operations everywhere across the image.

Convolutional neural networks (CNN)



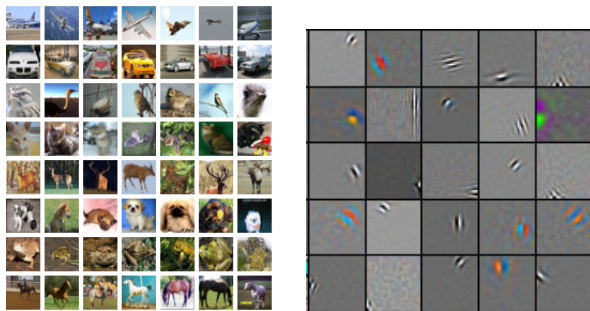
- ▶ Two main ideas:
 1. Use only **local** features.
 2. Apply **the same** features at many different positions.
- ▶ Add subsampling (max- or average-pooling) layers, so that higher layers can see (gradually) larger regions of the image.
- ▶ Standard reference: LeCun et al. 1998

Features *want* to be local



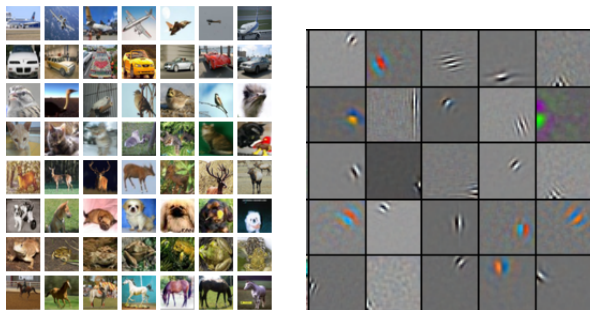
- (right) Features learned by a fully connected network on RGB images of size 32×32 pixels.

Features *want* to be local



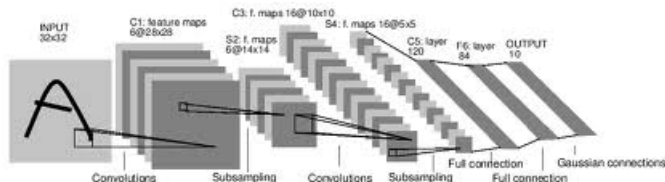
- ▶ (right) Features learned by a fully connected network on RGB images of size 32×32 pixels.
- ▶ A reasonable number of features for these images might be $32 \times 32 \times 3 = 3072$ (a complete basis)
- ▶ Better would be overcomplete.

Features *want* to be local



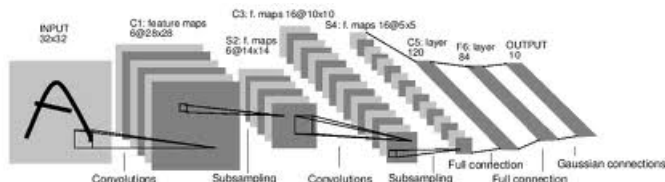
- ▶ (right) Features learned by a fully connected network on RGB images of size 32×32 pixels.
- ▶ A reasonable number of features for these images might be $32 \times 32 \times 3 = 3072$ (a complete basis)
- ▶ Better would be overcomplete.
- ▶ $\rightarrow$ you should use ≥ 9 mio parameters in the first layer

Convolutional neural networks (CNN)



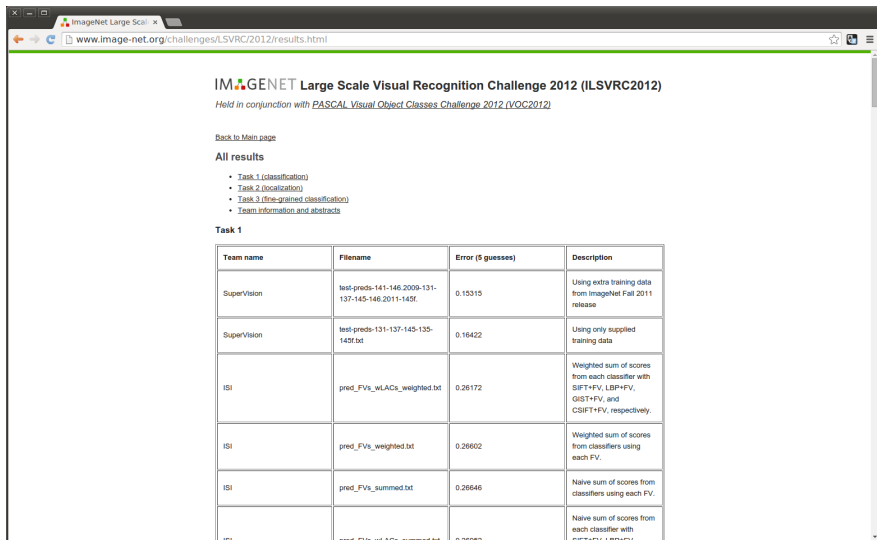
- ▶ A convolution layer operates on 3-D tensors:
 $\text{num vertical pixels} \times \text{num horizontal pixels} \times \text{num channels}$.
- ▶ The number of channels in the input can be one (gray value image), 3 (RGB image), or anything else. In higher layers it is usually more.

Convolutional neural networks (CNN)



- ▶ The number of parameters in a convolutional layer is:
 $\text{num vertical filtersize} \times \text{num horizontal filtersize} \times \text{num input channels} \times \text{num output channels}$
- ▶ (Without weight sharing, this could be eg.
 $10 \times 10 \times 3 \times 3072 \approx 1 \text{ mio (complete) }$)
- ▶ With weight sharing: $10 \times 10 \times 3 \times 100 \approx 30\text{k}$ (already overcomplete)
- ▶ The number of *hidden units* with weight sharing:
 $\text{num vertical pixels} \times \text{num horizontal pixels} \times \text{num channels}$ (eg. $32 \times 32 \times 100 = 10240$)

ImageNet challenge 2012



ImageNet Large Scale Visual Recognition Challenge 2012 (ILSVRC2012)

Held in conjunction with *PASCAL Visual Object Classes Challenge 2012 (VOC2012)*

[Back to Main page](#)

All results

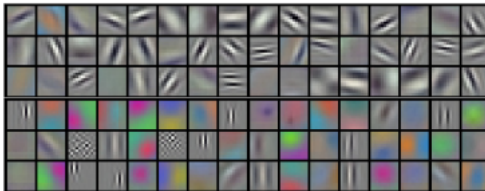
- [Task 1 \(classification\)](#)
- [Task 2 \(localization\)](#)
- [Task 3 \(fine-grained classification\)](#)
- [Team information and abstracts](#)

Task 1

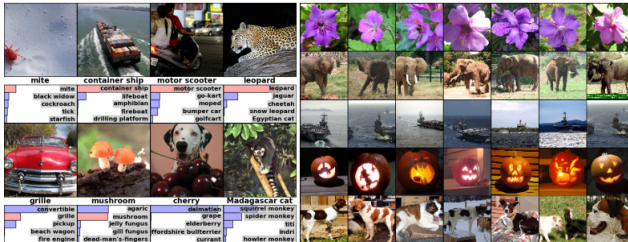
Team name	Filename	Error (5 guesses)	Description
SuperVision	test-preds-141-146.2009-131-137-145-146.2011-145f	0.15315	Using extra training data from ImageNet Fall 2011 release
SuperVision	test-preds-131-137-145-135-143f.txt	0.16422	Using only supplied training data
ISI	pred_FVs_wLACs_weighted.txt	0.26172	Weighted sum of scores from each classifier with SIFT+FV, LBP+*FV, GIST+*FV, and CSIFT+*FV, respectively.
ISI	pred_FVs_weighted.txt	0.26602	Weighted sum of scores from classifiers using each FV.
ISI	pred_FVs_summed.txt	0.26646	Naive sum of scores from classifiers using each FV.
ISI	pred_FVs_wLACs_summed.txt	0.26682	Naive sum of scores from each classifier with SIFT+*FV, LBP+*FV, GIST+*FV, and CSIFT+*FV, respectively.

ImageNet challenge 2012

some first-layer features



some results



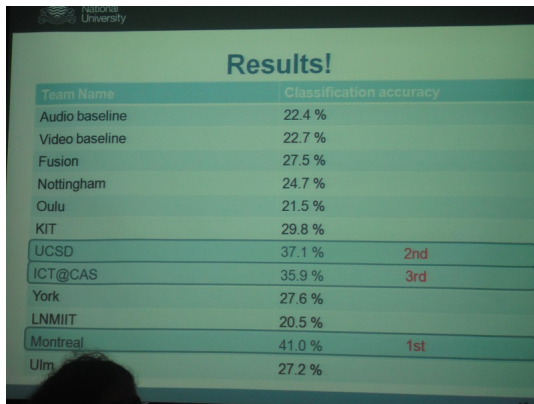
Krizhevsky, Sutskever, Hinton; 2012

GoogLeNet (Szegedy et al. 2014)



- ▶ Won ImageNet 2014 with **6.66%** top-5 error rate
- ▶ Key insights: Scaling up, unconventional architecture (eg. cross-channel pooling), intermediate targets
- ▶ A variation of this network based on *Batch Normalization* (Ioffe, Szegedy, 2015) achieves **4.8%** top-5 error rate, surpassing the accuracy of human labelers

Emotion recognition in the wild Challenge 2013

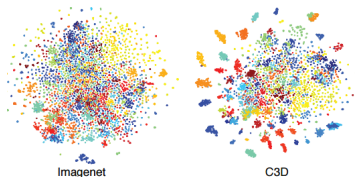


Team Name	Classification accuracy
Audio baseline	22.4 %
Video baseline	22.7 %
Fusion	27.5 %
Nottingham	24.7 %
Oulu	21.5 %
KIT	29.8 %
UCSD	37.1 % 2nd
ICT@CAS	35.9 % 3rd
York	27.6 %
LNMIIT	20.5 %
Montreal	41.0 % 1st
Ulm	27.2 %

- The biggest challenge in video: The amount of data.

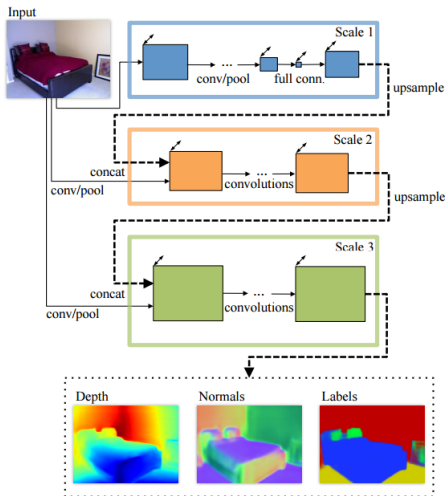
Video convnets: C3D

Dataset Task	Sport1M action recognition	UCF101 action recognition	ASLAN action similarity labeling	YUPENN scene classification	UMD scene classification	Object object recognition
Method	[19]	[39]([26])	[31]	[10]	[10]	[32]
Result	80.2	75.8 (89.1)	68.7	96.2	77.7	12.0
C3D	85.2	85.2 (90.4)	78.3	98.1	87.7	22.3
Δ	5.0	9.4 (1.3)	9.6	1.9	10.0	10.3



(Tran, Bourdev, Fergus, Torresani, Paluri; 2015)

Predicting depth, surface normals, semantic labels



Eigen, Fergus (2014)

High-level features

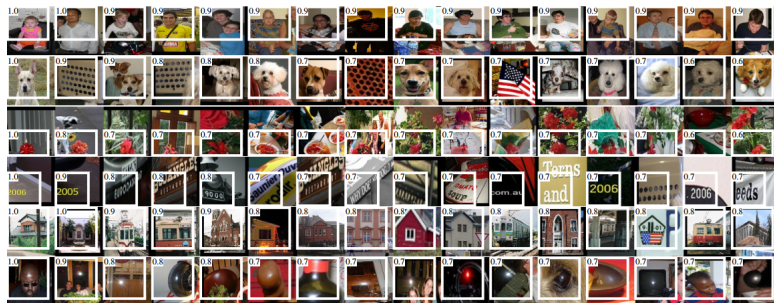
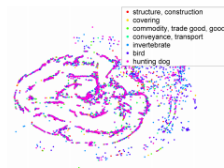


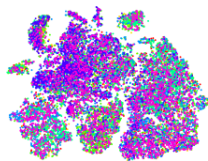
Figure 4: Top regions for six pool₅ units. Receptive fields and activation values are drawn in white. Some units are aligned to concepts, such as people (row 1) or text (4). Other units capture texture and material properties, such as dot arrays (2) and specular reflections (6).

Girshick et al., 2014

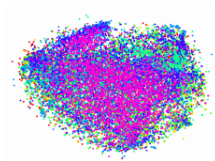
Conv-nets learn good *generic* features



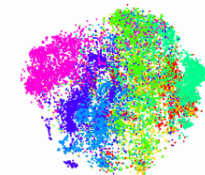
(a) LLC



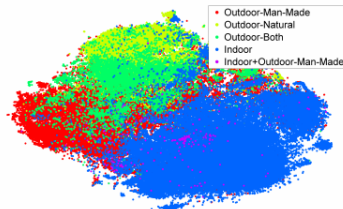
(b) GIST



(c) DeCAF₁



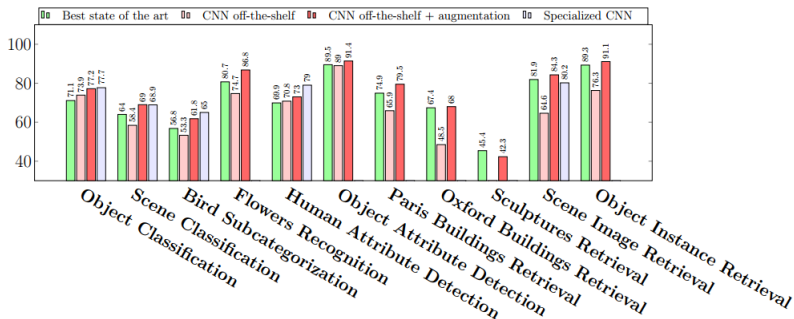
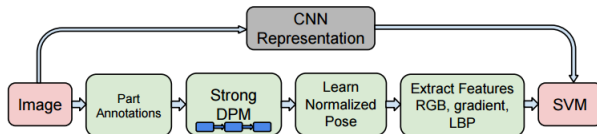
(d) DeCAF₆



non-Imagenet classes:

(Donahue et al, 2013)

Conv-nets learn good *generic* features

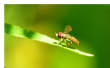


(Razavian, Azizpour, Sullivan, Carlsson; 2014)

Karayev et al 2014: Recognizing Image Style



HDR



Macro



Baroque



Roccoco



Vintage



Noir



Northern Renaissance



Cubism



Minimal



Hazy



Impressionism



Post-Impressionism



Long Exposure



Romantic



Abs. Expressionism



Color Field Painting

Flickr Style: 80K images covering 20 styles.

Wikipaintings: 85K images for 25 art genres.

Karayev et al 2014: Recognizing Image Style



Query: "dress".

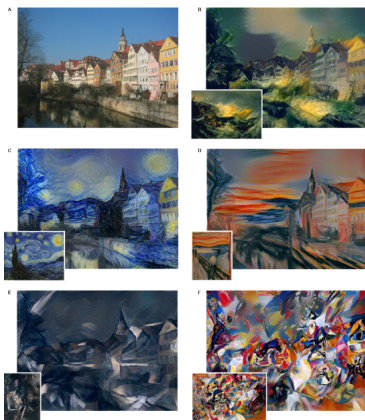


Query: "flower".

Off-the-shelf networks

- ▶ It has become hard to compete with features from a pre-trained network (such as GoogLeNet) on almost any recognition task.
- ▶ As a result, designing and training your own conv-net (similarly language-, translation-, speech-model, etc.) is becoming harder and less desirable.
- ▶ Pre-trained, off-the-shelf networks seem therefore likely to become much more important in the future.
- ▶ The upside: this leads to a separation-of-concerns that gives AI researchers a huge advantage over evolution.

Rendering images using a pre-trained convnet



Gatys, Ecker, Bethge (2015)

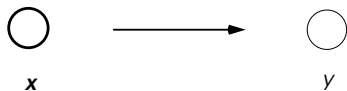
- Idea: optimize pixels so as to (i) match the hidden layer activations of the content image (ii) match a non-linear function of the hidden layer activations of the style image

Gradient descent on the image is a recurrence like DRAW

- ▶ Performing gradient descent on the input pixels is a recurrence very similar to DRAW (Gregor et al., 2015):
- ▶ The CNN is like an “encoder network” which computes hidden unit activations and compares these with the content/style images.
- ▶ Backpropagating the activation errors down to the pixels is equivalent to using a linear “decoder network” that writes the appropriate updates into the image buffer.
- ▶ The main difference (and opportunity for improving models like DRAW and LAPGAN) is that the encoder (and to some degree the decoder) is a pretrained conv-net which is already as good a visual representation as it gets (why learn it from scratch?)

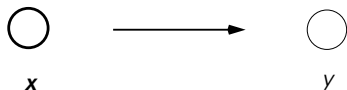
Structured prediction

Prediction:

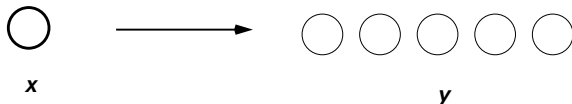


Structured prediction

Prediction:

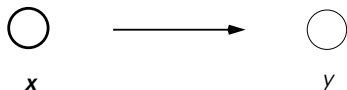


Structured prediction:

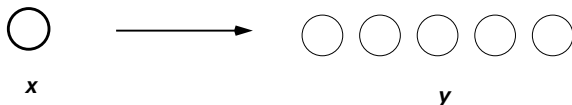


Structured prediction

Prediction:



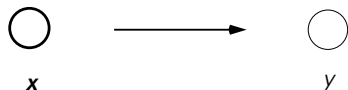
Structured prediction:



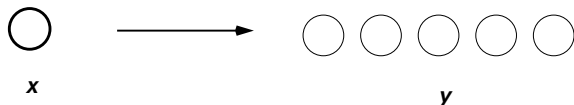
Problem: combinatorial explosion

Structured prediction

Prediction:

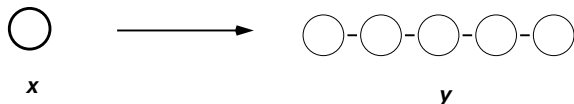


Structured prediction:



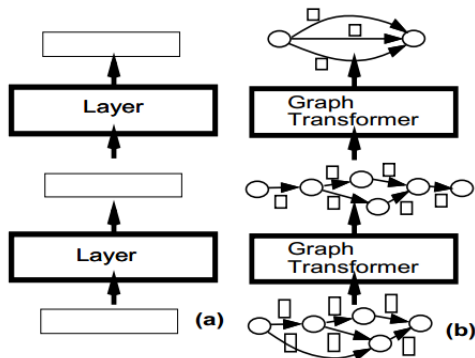
Problem: combinatorial explosion

Solution: Impose tractable dependency structure



- Applications: Scene labeling, text, speech, ...

Solution proposed in 1998



- ▶ Main insight: When layers are complex *graphs*, back-prop still works (LeCun et al 1998)
- ▶ This observation was recycled in 2001 under the name *Conditional Random Field*

Linear structured prediction

- ▶ Main idea: Define scores over cliques in a graph
- ▶ Example: sequence models may define a score

$$c(\mathbf{x}, \mathbf{y}) = \mathbf{w}^T \phi(\mathbf{x}, \mathbf{y}) = \sum_t \mathbf{w}^T \phi(\mathbf{x}, y_{t-1}, y_t)$$

- ▶ Allows using dynamic programming for computing costs and derivatives
- ▶ Extension to non-sequence models straightforward

Structured back-prop

- ▶ The reason for decomposition of cost/gradients is not linearity but additivity.
- ▶ We can use a neural network score

$$c(\mathbf{x}, \mathbf{y}) = \sum_t f_{\mathbf{w}}(\phi_t(\mathbf{x}, y_{t-1}, y_t))$$

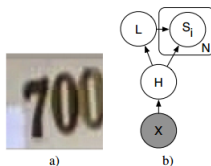
- ▶ The log-likelihood then still decouples

$$\begin{aligned} L(\mathbf{w}) &= \sum_i \sum_t f_{\mathbf{w}}(\phi(\mathbf{x}^i, y_{t-1}^i, y_t^i)) \\ &\quad - \log \sum_{\mathbf{y}} \exp\left(\sum_t f_{\mathbf{w}}(\phi(\mathbf{x}^i, y_{t-1}, y_t))\right). \end{aligned}$$

- ▶ ...and so does the gradient:

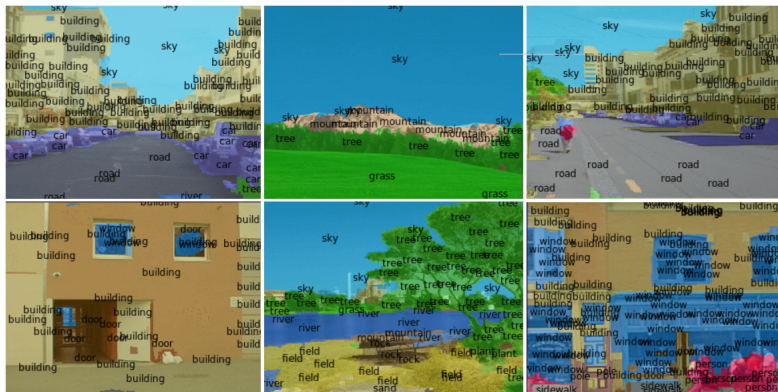
$$\frac{\partial L}{\partial \mathbf{w}} = \sum_i \sum_{\mathbf{y}} (\delta_{\mathbf{y}, \mathbf{y}^i} - p(\mathbf{y} | \mathbf{x}^i)) \sum_t \frac{\partial f_{\mathbf{w}}}{\partial \mathbf{w}}(\phi(\mathbf{x}^i, y_{t-1}^i, y_t^i))$$

Streetview (Goodfellow et al, ICLR 2014)



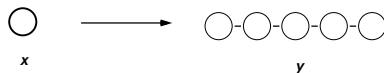
- see also: Jaderberg et al. ICLR 2015 for recognizing text in images

Towards scene understanding



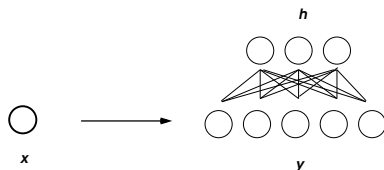
Farabet et al, 2013

A chance for unsupervised learning



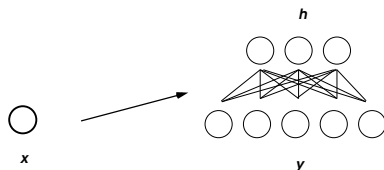
- ▶ A possible alternative to complicated graphical models on the outputs is to use *feature learning* on the outputs themselves.

A chance for unsupervised learning



- ▶ A possible alternative to complicated graphical models on the outputs is to use *feature learning* on the outputs themselves.

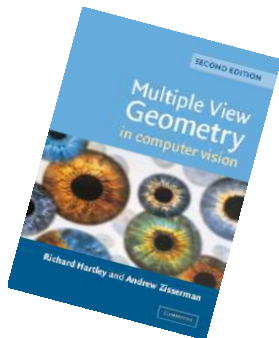
A chance for unsupervised learning



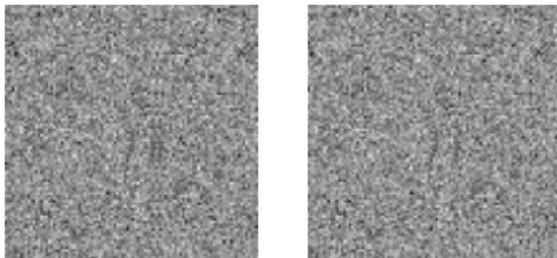
- ▶ A possible alternative to complicated graphical models on the outputs is to use *feature learning* on the outputs themselves.
- ▶ (...and potentially the dependence of the features themselves on the inputs, which would require 3-way connections)

Beyond recognition: Multiview learning

- ▶ Many vision (and other cognition) tasks depend on encoding relations across multiple images:
- ▶ Geometry, stereo, structure-from-motion, motion understanding, activity analysis, tracking, optical flow, modeling object relations, articulation, odometry, analogy, ...



Random dot stereograms



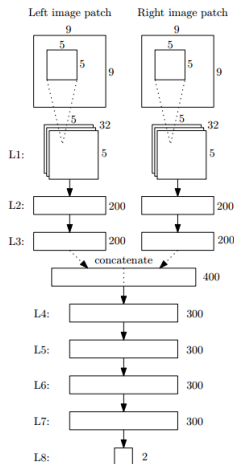
Some things are hard to infer from still images



There are things images cannot teach you

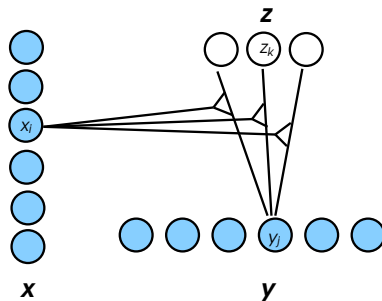


Deep learning for stereo matching



Best performing method on KITTI benchmark (2014)
Zbontar, LeCun (2014)

Learning transformations with bi-linear models

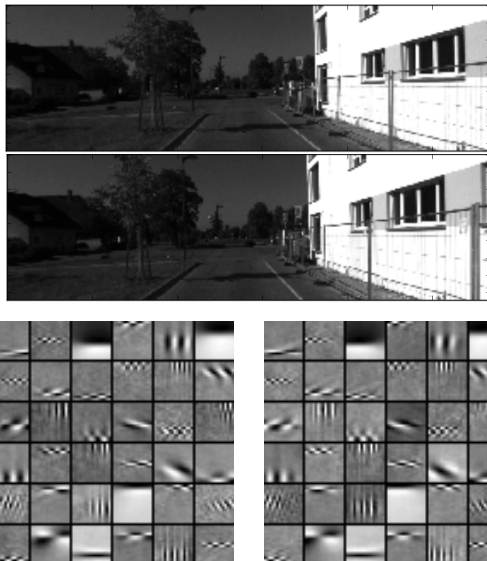


► $w_{jk}(\mathbf{x}) = \sum_i w_{ijk} x_i$, so

$$z_k = \sum_j w_{jk} y_j = \sum_j \left(\sum_i w_{ijk} x_i \right) y_j = \sum_{ij} w_{ijk} x_i y_j$$

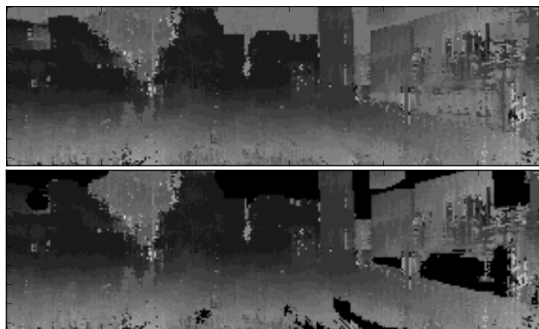
(Tenenbaum, Freeman; 2000), (Grimes, Rao; 2005),
(Olshausen; 2007), (Memisevic, Hinton; 2007)

Learning stereo vision with a bi-linear model



(Konda et al. 2013)

Inferred depth map

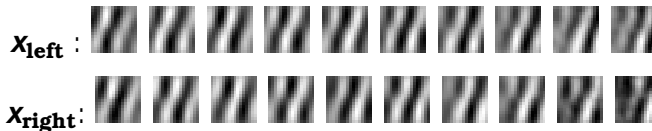


- Top: inferred depth map, bottom: thresholded to remove uncertain regions

Learning depth and motion: Hollywood 3D



► filters:



Precision rates by action class

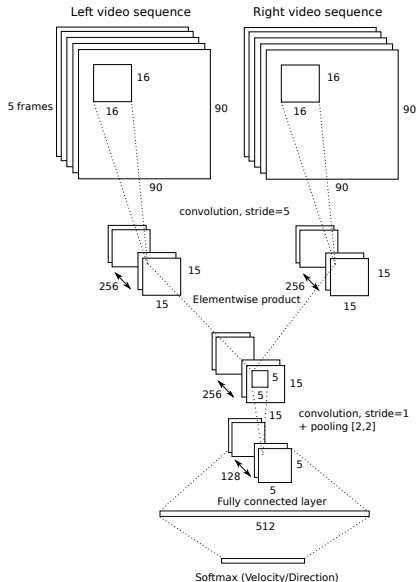
Action	SAE-MD	SAE-MD(Av)	SAE-MD(Ct)	SAE-M	SAE-D	ISA	3D-Ha	4D-Ha	3.5D-Ha
NoAction	12.10	12.77	13.10	15.73	12.15	12.27	12.1	12.9	13.7
Run	52.56	50.44	51.45	45.38	56.07	24.91	19.0	22.4	27.0
Punch	41.09	38.01	32.68	33.86	36.17	31.17	10.4	4.8	5.7
Kick	9.41	7.94	6.86	6.63	11.84	9.96	9.3	4.3	4.8
Shoot	30.26	35.51	30.49	30.52	40.72	32.48	27.9	17.2	16.6
Eat	5.85	7.03	6.78	7.29	9.03	6.89	5.0	5.3	5.6
Drive	52.65	59.62	51.35	61.61	45.19	54.47	24.8	69.3	69.6
UsePhone	22.79	23.92	19.01	23.60	23.36	17.67	6.8	8.0	7.6
Kiss	15.03	16.40	16.12	17.86	17.06	14.94	8.4	10.0	10.2
Hug	6.64	7.02	7.61	7.38	9.27	9.48	4.3	4.4	12.1
StandUp	37.35	34.23	37.01	29.16	15.01	26.71	10.1	7.6	9.0
SitDown	6.51	6.95	7.53	7.40	9.06	5.13	5.3	4.2	5.6
Swim	16.58	29.48	17.60	29.45	26.70	16.09	11.3	5.5	7.5
Dance	43.15	36.26	44.59	29.64	25.12	53.72	10.1	10.5	7.5
mean	25.14	26.11	24.45	24.61	24.05	22.55	12.6	13.3	14.1
AP									

Depth, motion, odometry



Images taken from KITTI odometry dataset and <http://www.cvlibs.net/datasets/kitti/index.php>

Depth, motion, odometry



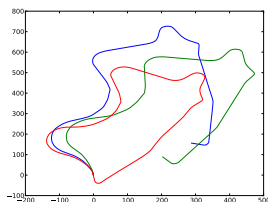
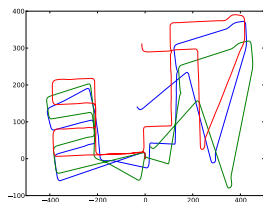
Konda, Memisevic (2014/2015)

Yoshua Bengio & Roland Memisevic

Deep Learning in Image Processing

September 27, 2015

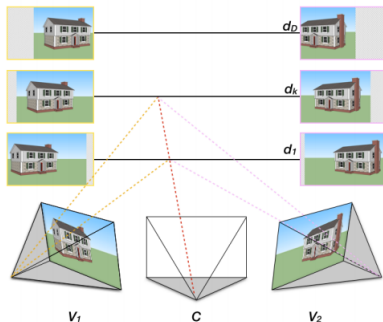
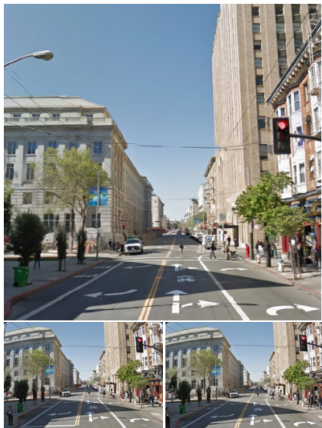
Depth, motion, odometry



(d) Train sequence (Seq.8) (e) Test sequence (Seq.9)

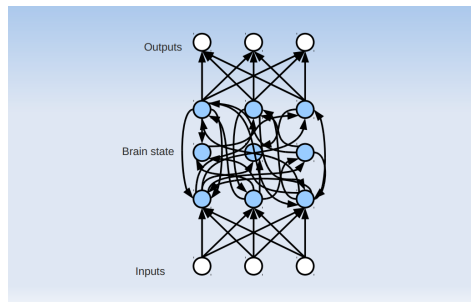
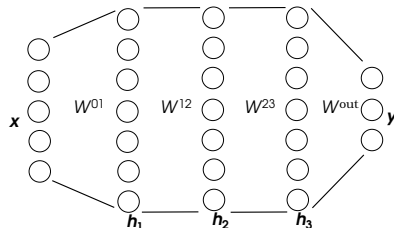
Ground truth shown in **red**, discretised ground truth in **green**, prediction in **blue**.
(Need landmark detection/loop closure techniques to improve maps.)

DeepStereo: stereo without explicit depth



Flynn, Neulander, Philbin, Snavely (2015)

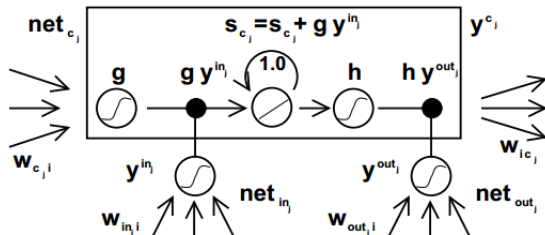
Recurrent networks



RNN picture from <http://www.cs.toronto.edu/~asamir/cifar/Ilya.slides.pdf>

- ▶ Stepping the network T time steps yields the equivalent of a T -layer feedforward net with weights that are shared between layers.
- ▶ Training the network by unrolling it in time is called back-prop-through-time (BPTT).
- ▶ Vanishing gradients especially problematic here.

Long-Short Term Memory (LSTM)



(Hochreiter, Schmidhuber; 1997)

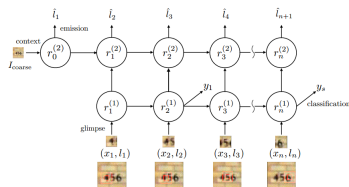
Caption generation

- An obvious application for RNNs in vision is to let them take care of the language side: Caption generation
- But nearest neighbor based methods perform surprisingly well (Devlin et al 2015)

Image	Selected Caption (BLEU)	Selected Caption (CIDEr)	Image	Selected Caption (BLEU)	Selected Caption (CIDEr)
	A bedroom with a bed and a couch.	A hotel room with two beds and a table.		A man riding a wave on a surfboard.	A man riding a wave on a surfboard in the ocean.
	A train is stopped at a train station.	A red and white train parked in a train station.		A person flying a kite in the sky.	A person flying a kite in the sky.
	A group of people sitting around in a living room.	A group of people sitting on a couch in a living room.		A cat sitting in a bathroom sink.	A black and white cat sitting in a bathroom sink.
	A group of people washing elephants in the water.	An elephant is swimming in the water near the rocks.		A wooden bench in front of a building.	A wooden bench in front of a building.
	Two zebras and a giraffe in a field.	Two zebras and a giraffe in a field.		A baby elephant is standing in a field.	An elephant is walking through a grassy field.

Visual attention

Hard attention

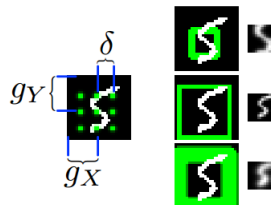


Ba et al. 2015, Mnih et al. 2013

Challenge: Marginalize over fixation sequence

Model	Test Err.
11 layer CNN Goodfellow et al. (2013)	3.96%
10 layer CNN	4.11%
Single DRAM	5.1%
Single DRAM MC avg.	4.4%
forward-backward DRAM MC avg.	3.9%

Soft attention

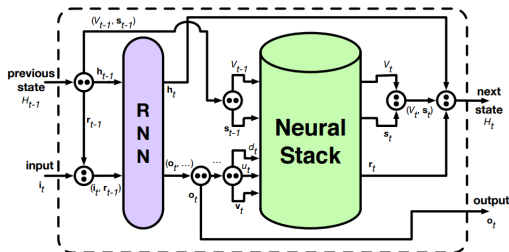


Gregor et al., 2015

Challenge: computational (go through whole image at every time-step)

Towards differentiable models of computation

- ▶ Neural Turing Machine (Graves et al, 2014)
- ▶ Memory Networks (Weston et al, 2014)
- ▶ Learning to Transduce with Unbounded Memory (Grefenstette et al. 2015)

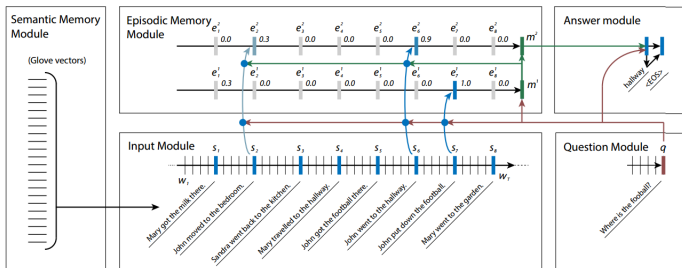


- ▶ To be able to back-propagate, all operations are based on differentiable operations
- ▶ But sampling-based methods may work here, too

From neural networks to “neural programs”

I: Jane went to the hallway.
 I: Mary walked to the bathroom.
 I: Sandra went to the garden.
 I: Daniel went back to the garden.
 I: Sandra took the milk there.
 Q: Where is the milk?
 A: garden

I: Everybody is happy.
 Q: What's the sentiment?
 A: positive
 Q: What are the POS tags?
 A: NN VBZ JJ .
 I: The answer is far from obvious.
 Q: In French?
 A: La réponse est loin d'être évidente.



Kumar et al 2015

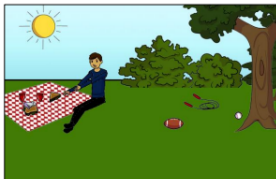
From neural networks to “neural programs”



What color are her eyes?
What is the mustache made of?



How many slices of pizza are there?
Is this a vegetarian pizza?



Is this person expecting company?
What is just under the tree?



Does it appear to be rainy?
Does this person have 20/20 vision?

Visual Question Answering (Antol et al, 2015)

Von Neumann via Deep Learning

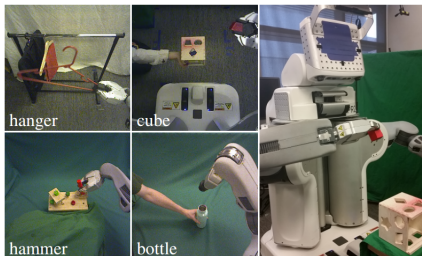
- ▶ In the past we simulated neural nets on classic hardware and it didn't work
- ▶ Today we simulate classic hardware on neural nets and it works

The benefit of today's way: Add more parallel hardware and your program will run faster.

Von Neumann via Deep Learning in Vision?

- ▶ In many tasks, allowing a model to revise its response rather than asking it to compute it in one pass seems like a good idea.
- ▶ detection?
- ▶ structured prediction?
- ▶ segmentation?
- ▶ image synthesis?
- ▶ generating mocap sequences?
- ▶ recognition?

Robotics/RL



Levine et al. 2015

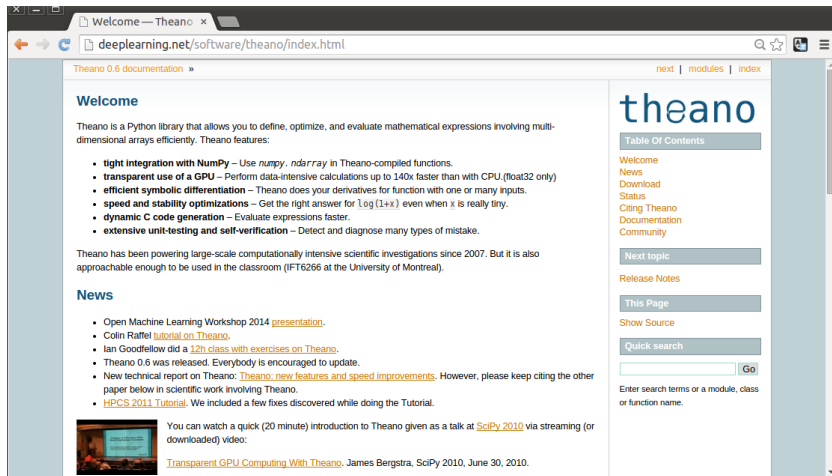


Mnih et al. 2013

- Removing vision as a proper subtask?

Deep Learning Software/Frameworks

- ▶ Back-prop: torch, theano
- ▶ Add-ons: blocks, fuel, lasagne
- ▶ Low-level: cuDNN, nervanagpu, cudamat
- ▶ Convnets: caffe, overfeat, cuda-convnet, sklearn-theano



<http://www.deeplearning.net/tutorial/>

theano code snippet (from deeplearning.net)

```
import theano
from theano import tensor

a = tensor.dscalar()
b = tensor.dscalar()

c = a + b

f = theano.function([a,b], c)

assert 4.0 == f(1.5, 2.5)
```

theano code snippet: linear regression

```
import theano
import theano.tensor as T

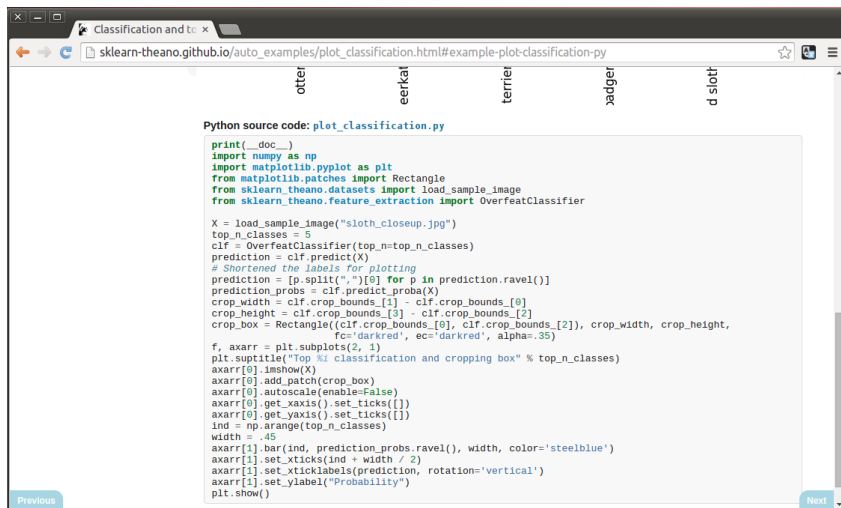
#define computational graph:
W = T.dmatrix()
inputs = T.dmatrix()
targets = T.dmatrix()
outputs = T.dot(W.T, inputs)
cost_theano = ((outputs - targets)**2).mean()
grad_theano = T.grad(cost_theano, W)

#compile functions:
cost = theano.function([W,inputs,targets], cost_theano)
grad = theano.function([W,inputs,targets], grad_theano)

#try on some _random_ data:
my_w = 0.01*randn(10,1).astype("float32")
my_inputs = randn(100,10).T.astype("float32")
my_targets = randn(1,100).astype("float32")
print cost(my_w, my_inputs, my_targets)
my_w -= 0.1*grad(my_w, my_inputs, my_targets)
print cost(my_w, my_inputs, my_targets)
```

theano code snippet: autoencoder

```
.  
.   
.   
prehiddens = T.dot(inputs, W)  
hiddens = (prehiddens > selectionthreshold) * prehiddens  
outputs = T.dot(hiddens, W.T) + bvis  
  
cost = T.mean(T.sum(0.5 * ((inputs - outputs)**2), axis=1))  
grad = T.grad(cost, params)  
.   
.   
. 
```



The screenshot shows a web browser window with the address bar displaying `sklearn-theano.github.io/auto_examples/plot_classification.html#example-plot-classification-py`. The browser window contains a plot of a sloth image with five predicted class labels: 'otter', 'eekat', 'terrier', 'xader', and 'd sloth'. Below the plot, the Python source code for `plot_classification.py` is displayed. The code imports necessary libraries, loads a sample image, uses an `OverfeatClassifier` to predict classes, and generates a bar chart showing the top 5 predicted classes and their probabilities. The plot also includes a cropping box around the image.

```
Python source code: plot_classification.py

print(__doc__)
import numpy as np
import matplotlib.pyplot as plt
from matplotlib.patches import Rectangle
from sklearn_theano.datasets import load_sample_image
from sklearn_theano.feature_extraction import OverfeatClassifier

X = load_sample_image("sloth_closeup.jpg")
top_n_classes = 5
clf = OverfeatClassifier(top_n=top_n_classes)
prediction = clf.predict(X)
# Shortened the labels for plotting
prediction = [p.split(",")[0] for p in prediction.ravel()]
prediction_probs = clf.predict_proba(X)
crop_width = clf.crop_bounds[1] - clf.crop_bounds[0]
crop_height = clf.crop_bounds[3] - clf.crop_bounds[2]
crop_box = Rectangle((clf.crop_bounds[0], clf.crop_bounds[2]), crop_width, crop_height,
                    fc='darkred', ec='darkred', alpha=.35)

f, axarr = plt.subplots(2, 1)
plt.suptitle("Top %i classification and cropping box" % top_n_classes)
axarr[0].imshow(X)
axarr[0].add_patch(crop_box)
axarr[0].autoscale(enable=False)
axarr[0].get_xaxis().set_ticks([])
axarr[0].get_yaxis().set_ticks([])
ind = np.arange(top_n_classes)
width = .45
axarr[1].bar(ind, prediction_probs.ravel(), width, color='steelblue')
axarr[1].set_xticks(ind + width / 2)
axarr[1].set_xticklabels(prediction, rotation='vertical')
axarr[1].set_ylabel("Probability")
plt.show()
```

Gatys et al. using sklearn-theano

```
BLIND IMAGE DATA
contentimage = getimage("./contentimage.png")
styleimage = getimage("./styleimage.jpg")

#GET CONVNET LAYERS IN THE FORM OF THEANO EXPRESSIONS:
theanolayers, input_var = sklearn_theano.feature_extraction.caffe.googlenet.create_theano_expressions()
expressions = [theanolayers['conv1/x7_s2'], theanolayers['conv2/x3'], theanolayers['inception_3a/output'], theanolayers['inception_3b/output'], theanolayers['inception_4a/output'], theanolayers['inception_4b/output'], theanolayers['inception_4d/output'], theanolayers['inception_4e/output']]
contentweights, styleweights = [0.0, 0.0, 0.001, 0.0, 0.001, 0.0, 0.0, 0.0, 0.0, 0.0], [1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0, 1.0]

styleweights = [theano.shared(value=w, name=str(i)) for i, w in enumerate(styleweights)]
contentweights = [theano.shared(value=w, name=str(i)) for i, w in enumerate(contentweights)]

#EXTRACT LAYER ACTIVATIONS AND DEFINE CONTENT AND STYLE COSTS:
totalcost = 0.0
layer_content_cost = []
layer_style_cost = []

for layerindex, expression in enumerate(expressions):
    layeroutput_function = theano.function([input_var], expression)
    contentimage_layer = layeroutput_function(contentimage)
    styleimage_layer = layeroutput_function(styleimage)

    #DEFINE COST AND GRADIENT FOR CONTENT:
    layer_content_cost.append(((expression-contentimage_layer)**2).sum())

    #DEFINE COST AND GRADIENT FOR STYLE (WE USE SCAN TO ITERATE OVER THE ROWS OF THE FEATUREMAP, OTHER ATTEMPTS LED TO RECURSION_DEPTH ISSUES):
    gramian_original = numpy.zeros((styleimage_layer.shape[1], styleimage_layer.shape[1]), dtype="float32")
    for i in range(styleimage_layer.shape[1]):
        for j in range(styleimage_layer.shape[1]):
            gramian_original[i,j] = (styleimage_layer[0,i,:]* styleimage_layer[0,j,:]).sum(1).sum(0)

    gramian_testimage, updates = theano.scan(fn=lambda onefeatureplane, featuremap: (onefeatureplane.dotshuffle('x',0,1)*featuremap).sum(2).sum(1),
        outputs_info=None,
        sequences=[expression[0,:,:,1]],
        non_sequences=[expression[0,i,i,1]])

    layer_style_cost.append(((gramian_testimage - gramian_original)**2).sum()) / (2*(styleimage_layer.shape[2]*styleimage_layer.shape[3])**2 * (styleimage_layer.shape[1])**2)

#DEFINE TOTAL COST AS WEIGHTED SUM OF CONTENT AND STYLE COST
totalcost += contentweights[layerindex] * layer_content_cost[layerindex] + styleweights[layerindex] * layer_style_cost[layerindex]

totalgrad = theano.grad(totalcost, input_var)

#COMPILE THEANO FUNCTIONS:
cost = theano.function([input_var], totalcost)
grad = theano.function([input_var], totalgrad)

#TRY IT OUT:
inout = contentimage.copy()
stepsize = 0.1
momentum = 0.9
inc = 0
for i in range(10000):
    inc = momentum * inc - stepsize * grad(inout)
    inout += inc
    inout[inout<10] = 10
    inout[inout>245] = 245
    print cost(inout)

#SHOW RESULT:
showin(inout)
```

64,1

Top

Outlook/directions

- ▶ More applications
- ▶ Architectures (attention, vanishing gradients, **neural programs**)
- ▶ Reinforcement learning
- ▶ Theory
- ▶ Scaling up, **hardware**
- ▶ Multimodality and **grounding**: vision, language, speech, robotics

Thank you
Questions?